

Placement Portal V2

Modern Application Development II — Project Report

Author

Full name	Vaibhav Tanwar
Roll number	23f2005318
Student email	23f2005318@ds.study.iitm.ac.in
About me	I am a Final Term Diploma-Level student at this Online Degree. And I am passionate about full stack web-development

Description

A multi-user campus placement platform for an institute placement cell. Companies publish placement drives, students find the drives they are eligible for and apply, and the placement cell approves companies and drives, blacklists bad actors and monitors placement statistics. Scheduled background jobs send students daily reminders and email the cell a monthly placement report.

Technologies used

- **Flask 3.1** — serves the REST API and the entry-point page; chosen for its small surface area and blueprint-based modularity.
- **Flask-SQLAlchemy 3.1 with SQLite** — ORM over a single database file at instance/placement.sqlite3. Models declare relationships and cascades, so referential integrity is enforced in one place and no database server is needed.
- **Flask-JWT-Extended 4.7 and Werkzeug security** — stateless tokens with a 12-hour expiry and a role claim, so no server-side session store is required; passwords are stored only as salted hashes.
- **Flask-Caching 2.4 and Celery 5.6, both on Redis** — Redis caches read-heavy endpoints for 60 seconds and falls back to in-process caching if it is down, and also acts as Celery's broker, result backend and beat scheduler, keeping slow email work off the request thread.
- **Vue 3 and Vue Router 4 via CDN, with Jinja2** — a reactive single-page UI with no build step. Jinja2 renders exactly one template, index.html, purely as the SPA entry point; no interface is built with it.
- **Bootstrap 5.3 and Bootstrap Icons** — the only CSS framework used; responsive grid, tables, forms and badges out of the box.

DB Schema Design

Table	Columns	Keys and constraints
users	id, email, password_hash, role (admin company student), is_active, created_at	PK id; email UNIQUE NOT NULL and indexed; password_hash and role NOT NULL
students	id, user_id, full_name, roll_number, branch, cgpa, grad_year, phone, resume_url, about, is_blacklisted, created_at	PK id; FK user_id → users.id, UNIQUE (one profile per login); roll_number UNIQUE; full_name NOT NULL
companies	id, user_id, name, hr_contact, hr_phone, website, location, industry, overview, logo_url, approval_status (pending approved rejected), is_blacklisted, created_at	PK id; FK user_id → users.id, UNIQUE; name NOT NULL; approval_status defaults to pending
drives	id, company_id, drive_name, job_title, job_description, eligible_branches, min_cgpa, eligible_year, salary, location, openings, interview_mode, application_deadline, status (pending approved closed rejected), created_at	PK id; FK company_id → companies.id ON DELETE CASCADE; drive_name and job_title NOT NULL; status defaults to pending
applications	id, student_id, drive_id, applied_on, status (applied shortlisted waiting selected rejected), remark, interview_datetime, updated_at	PK id; FK student_id → students.id and drive_id → drives.id, both CASCADE; UNIQUE (student_id, drive_id)

Design rationale. Authentication is separated from profile data: one users table holds the credentials and role for all three roles, while students and companies hold role-specific attributes. This keeps login logic uniform and lets the admin deactivate any account through a single is_active flag. Eligibility criteria live on the drive because the wireframe treats them as fixed attributes; branches are a comma-separated string where an empty value means all branches. The

applications table joins students to drives, and a composite UNIQUE constraint on (student_id, drive_id) makes duplicate applications impossible at the database level rather than in application code. Foreign keys cascade on delete, so removing a company cleanly removes its drives and their applications.

API Design

Thirty-five operations across thirty-three paths form a JSON REST API under the /api prefix, organised into four Flask blueprints — auth, admin, company and student — with resources for users, students, companies, drives, applications and reports.

Resource	Representative endpoints	Access
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me	Public (except /me)
Companies	GET /api/admin/companies/{id}, POST /api/admin/companies/{id}/approval, POST /api/admin/companies/{id}/blacklist, PUT /api/company/profile	Admin; company owns its profile
Students	GET /api/admin/students/{id}, POST /api/admin/students/{id}/blacklist, PUT /api/student/profile	Admin; student owns its profile
Drives	POST /api/company/drives, PUT /api/company/drives/{id}, POST /api/admin/drives/{id}/approval, GET /api/student/drives	Company writes, admin approves, student reads
Applications	POST /api/student/drives/{id}/apply, DELETE /api/student/applications/{id}, PUT /api/company/applications/{id}, PUT /api/company/applications/bulk	Student applies, company decides
Reports	GET /api/admin/stats, POST /api/admin/reports/reminders, POST /api/admin/reports/monthly, GET /api/health	Admin; /health is public

Implementation. Every protected route carries a `role_required(*roles)` decorator that reads the role claim from the JWT and returns 403 before the handler runs, so a student can never reach a company endpoint by guessing the URL. Ownership is checked separately: a company may only read and update its own drives and applications. Status codes are meaningful throughout — 201 on creation, 202 when a Celery job is queued, 403 when eligibility fails, 409 for a duplicate application or invalid withdrawal. The full specification is submitted separately as `api.yaml` (OpenAPI 3.0.3).

Architecture and Features

Organisation. The project root holds `app.py` (application factory and entry point), `config.py`, `extensions.py`, `models.py`, `seed.py`, `mailer.py`, `celery_app.py` and `tasks.py`. Controllers live in the `api/` package as one blueprint per role (`auth.py`, `admin.py`, `company.py`, `student.py`), with shared helpers in its `__init__.py`; `templates/` holds the single Jinja2 entry point. The front end sits in `static/js/` as `api.js` (fetch wrapper that attaches the token), `store.js` (shared state and formatters), `components.js` (six reusable components), `router.js` (routes and role guards) and a `pages/` folder with one file per role. The database is created under `instance/`, and generated reports and saved emails are written to `reports/`.

Core features. The admin is a seeded superuser who approves or rejects company registrations and drives, searches students, companies and drives from one search box, blacklists or reinstates accounts, and views placement statistics. Companies register, wait for approval, then create drives, review applicants, shortlist them with remarks and interview details, record final selections and close or reopen drives. Students register, edit their profile, browse approved drives with an eligibility filter, apply, withdraw while still in the applied state, and follow results in a history table. Per the wireframe note, blacklisting a company also cancels all of its pending and approved drives.

Additional features. Redis caching on read-heavy endpoints, cleared on every write. Two Celery beat jobs — a daily 18:00 reminder to students eligible for an open drive who have not applied, and a monthly report on the 1st at 08:00 that builds an HTML placement summary and emails it to the cell — both also triggerable manually from the admin reports page. Every outgoing email is archived to `reports/outbox/` so the jobs can be verified without a mail server. Also: server-side eligibility re-validation on apply, a `/api/health` endpoint reporting database and Redis status, a seed script with a demo dataset, and a thirty-two-check smoke test suite in `test_api.py`.

Video

Video link	https://drive.google.com/drive/u/0/folders/1G1UaJbLOi4TMKVtDF-gszK50G4BU-b8D
------------	---